Bridge

Bridge Pattern

Split abstraction from implementation

DEFINITION

Decouple an abstraction from its implementation so the two can vary independently.

Bridge separates an abstraction from its implementation into two class hierarchies joined by composition: the abstraction holds an implementor and delegates to it, rather than extending it. Each side then subclasses independently — shapes on one tree, renderers on the other — and you can swap the implementation at runtime.

WHY IT MATTERS

When two dimensions vary together under one inheritance tree, you get a class explosion — m shapes by n renderers means m x n subclasses. Bridge gives each dimension its own tree, collapsing m x n into m + n, and lets the implementation be selected or switched at runtime. The cost is upfront indirection: with only one implementation it is needless ceremony.

Vasyl Krupka · Ukraine

How it works

Abstraction	High-level interface; holds a reference to an Implementor.
Implementor	The low-level interface — deliberately different from the Abstraction's.
ConcreteImplementor	A specific platform doing the primitive operations.

HOW TO APPLY

1. Spot the two independent dimensions (abstraction vs platform / detail).
2. Define an Implementor interface for the primitive operations.
3. Have the Abstraction hold an implementor and delegate to it (inject it via DI).
4. Subclass each tree on its own; pick or swap the implementor at runtime.

> LITMUS TEST
>
> Do two dimensions vary independently, threatening an m x n subclass explosion? If both an abstraction and its platform must extend separately, bridge them by composition.

— Smell → fix

One inheritance tree multiplies both dimensions — m x n subclasses:

```
class SvgCircle {} // shape x renderer in one tree
class CanvasCircle {}
class SvgSquare {} // add a shape or a renderer and
class CanvasSquare {} // the matrix doubles
```

↓ SPLIT INTO TWO TREES, BRIDGE BY COMPOSITION

```
interface Renderer { circle(r): void }
class Circle { // abstraction tree
r!: Renderer // the bridge
draw() { this.r.circle(10) } // delegate to impl
}
const c = new Circle(); c.r = new SvgRenderer()
c.draw() // swap r to CanvasRenderer → same Circle
```

Circle (abstraction) now holds a Renderer (implementor) and delegates; add a shape or a renderer on its own tree. m + n classes instead of m x n, and you can swap the renderer at runtime. In TS, inject the implementor — that is the bridge.

USE WHEN

- ✓ Two independent dimensions (shape x renderer)
- ✓ Avoid a class explosion of m x n

AVOID WHEN

- ▲ Only one dimension varies — YAGNI

— Trade-offs & pitfalls

PROS

+ m + n classes instead of m x n
+ Swap implementation at runtime

CONS

− Upfront indirection
− Harder to grasp

COMMON PITFALLS

▲ Over-engineering with only one implementation — the extra hierarchy is pure overhead until a second platform is real.

▲ Confusing it with Strategy: structurally identical, but Bridge separates a platform / impl, Strategy swaps an interchangeable algorithm.

▲ Forgetting it is two deliberately different interfaces — the Abstraction and Implementor shapes shouldn't collapse into one.

WHERE YOU SEE IT

- JDBC: one app API bridged to per-vendor DB drivers; SLF4J bridged to log4j / logback backends.
- Shapes by renderers, devices by remotes, messages by channels — any two independent dimensions.
- DI-injected implementations in TS / Nest: program the abstraction to an injected interface.

SEE ALSO

Strategy	same composition shape — Strategy swaps an algorithm, Bridge varies a platform / impl
Adapter	also composition, but retrofitted to fix a mismatch; Bridge is planned up front
Abstract Factory	can build the matching abstraction and implementor pairs
Dependency Injection	the modern way to wire the implementor into the abstraction

Interview & sources

Bridge vs Adapter?

Bridge is designed in beforehand so abstraction and implementation vary independently; Adapter is added after the fact to make an existing class fit.

Bridge vs Strategy?

The code looks the same — composition plus delegation. Bridge separates an abstraction from its platform / impl; Strategy swaps an interchangeable algorithm.

What problem does it solve?

The m x n subclass explosion: two independent dimensions get separate hierarchies, giving m + n classes instead of m x n.

Where is the bridge?

The reference the Abstraction holds to an Implementor — calls cross from the abstraction tree to the implementation tree there.

REMEMBER

Split an abstraction from its implementation into two trees joined by composition — both vary freely.

SOURCES

GoF — "Design Patterns" (1994): Bridge (structural)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Bridge (leftover patterns)